

Session 7 · Deck 7.B3

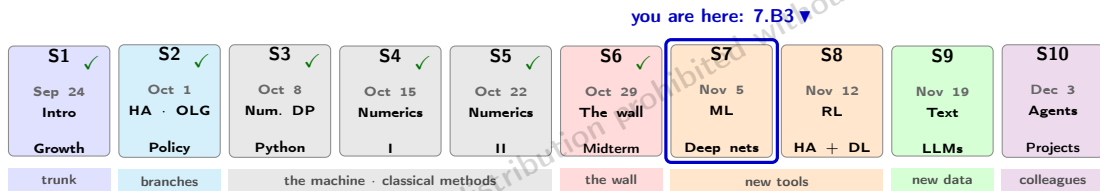
Structure, and How You Know It Worked

Zhigang Feng

Thursday, November 5, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 7.B1 the model as a loss function · 7.B2 deep VFI and actor-critic · **7.B3** structure and validation.

Two Debts

Block B has produced two working solvers and two unpaid debts.

First, the networks obey no economics. Theory says a value function is concave in capital and a consumption policy is increasing in wealth. Nothing in 7.B1 or 7.B2 imposes either. The network may satisfy them approximately where it was trained and violate them anywhere else — and a violated shape restriction is not a small error, it is a *qualitatively wrong* object.

Second, the *a priori* bound is gone. 3.A2 could bound the distance to the true value function **before looking at the answer**. Nothing in Block B can do that.

This deck pays both

Structure — build the restriction into the architecture, so it holds *identically* rather than approximately. 7.A4's **fourth** constraint class. **Validation** — replace the lost error bound with a measurement you can actually make.

The course's first deck said it: **a numerical solution is a claim**. This is where that claim gets adjudicated for a network.

Injecting Economic Structure

A generic feedforward network is *deliberately* assumption-free: 7.A4 called a dense $\mathbf{W}$ the refusal to assume anything about the input. For images that refusal costs parameters. **In economics it costs something worse**, because we are not ignorant — theory tells us the shape.

- $V(k)$ is **concave** in capital, from concave u and f ;
- $c(a)$ is **increasing** in wealth;
- a demand function is **monotone** in price;
- a cost function is **convex** in input prices.

Every one of these is a restriction that can be built into the parameterisation instead of hoped for.

7.A4's fourth class

CNN: $\mathbf{W}$ sparse and repeated. RNN: $\mathbf{W}$ shared across dates. GNN: $\mathbf{W}$ follows the adjacency. **Here: constrain the *signs* of the weights**, and the function is convex or monotone by construction.

And it is the same trade as ever. True, it buys parameters, data and reliability. False, it is bias — which is why the restriction must come from theory, not from convenience.

Measured: What an Unconstrained Network Does to Convexity

Fit a plain ReLU network (1,153 parameters) to a function that is **exactly convex** — $0.6x^2 + 0.25x + 0.1$, second derivative 1.2 everywhere — at **5 random seeds**.

	median RMSE	worst $-f''$
plain network	0.00172	5.93 (range 0–10.4)
ICNN	0.01345	$< 7.1 \times 10^{-11}$

“Worst $-f''$ ” is the **most negative curvature the fitted function reaches**: zero means it came out convex everywhere. At a typical seed the plain network fits the *level* well and is **locally concave by about** 5.9, against a true curvature of 1.2.

Fatal, not untidy

A concave patch breaks 7.B2’s inner maximisation: local maxima appear, ascent lands in the wrong one, the policy **jumps**. **And it is invisible in the loss.**

Read the range, not the median. At one seed in five the plain network came out convex anyway: **not reliably broken, but unreliable** — and you cannot tell which you have without checking. The ICNN never exceeds 7.1×10^{-11} . **Not free:** the ICNN’s RMSE is **7.8×** larger. *That is the price of the guarantee.*

Ledger icnn, five seeds. “Worst $-f''$ ” is the largest negative second difference; zero means convex. Code: [s07_generate.py](#), [run_icnn](#); L07 Part 6.

Input Convex Neural Networks

The architecture is a feedforward network with one extra channel and two restrictions:

$$z_{l+1} = \sigma_l \left(W_l^{(z)} z_l + W_l^{(x)} x + b_l \right).$$

Sufficient for $f(x)$ to be convex in x :

1. the **recurrent** weights are non-negative, $W_l^{(z)} \geq 0$;
2. the activations σ_l are **convex and non-decreasing** — ReLU and softplus both are.

The direct input weights $W_l^{(x)}$ are unconstrained, including negative: every layer sees x again through a free linear map, so the network can place and tilt the function freely.

Amos, Xu & Kolter (2017), *ICML*, "Input Convex Neural Networks".

The proof, by induction

Base. $z_0 = x$ is linear, hence convex. **Step.**

z_l convex; $W_l^{(z)} \geq 0$ makes $W_l^{(z)} z_l$ a **non-negative combination of convex functions**.

Adding the affine term preserves convexity, and so does applying a convex *non-decreasing* σ .

Both conditions are load-bearing: a non-monotone activation such as $\tanh$ breaks the induction at the last step.

Does the Constraint Cost Expressivity?

The obvious worry: with $W^{(z)} \geq 0$, is universal approximation lost?

Theorem (Chen, Shi & Zhang, 2019)

An input convex neural network can approximate **any convex function** on a compact domain to arbitrary accuracy.

The intuition is one line: a maximum of affine functions $\max_i \{a_i^\top x + b_i\}$ is convex, and *any* convex shape is a limit of such maxima. An ICNN with ReLU units is a hierarchical max-affine approximator.

The reading that matters

You are **not** imposing a functional form — not CES, not Cobb–Douglas, not translog. You are imposing a **shape restriction**, and keeping full non-parametric flexibility inside it. *Architecture encodes the theory; the data choose the function within it.*

Exactly the situation an economist is usually in: theory pins the *global shape* and says nothing about the curvature.

Chen, Shi & Zhang (2019), *ICLR*, "Optimal Control via Neural Networks: A Convex Approach".

And What the Constraint Refuses

Point the same ICNN at $\sin 3x$ on $[-1, 1]$, which is **not** convex, and train it identically.

ICNN on $\sin 3x$

RMSE, median of 5	0.3449
convexity violation	$< 8.9 \times 10^{-11}$

It fails, completely — and it fails while remaining **exactly convex**. The architecture did not bend; it could not.

This is the guarantee from the other side

A constraint that can be violated when the data push hard enough is not a guarantee.

This one cannot be, which is the entire point. The cost is stated exactly: if the true function is not convex, the ICNN will not find it, and the residual will say so loudly.

And that is a diagnostic worth having: **a structured network that cannot fit is telling you the theory you imposed is wrong** — an informative failure, unlike a silently wrong fit.

Ledger `icnn.nonconvex`; same architecture, optimizer and budget as the convex case. Code: `s07_generate.py`, `run_icnn`.

What Convexity Buys a Macro Solver

1. **The inner maximisation becomes convex.** 7.B2's obstacle was $\max_{k'} \{u(k, k') + \beta \hat{V}(k')\}$ with $\hat{V}$ a non-convex network. With $\hat{V}$ concave by construction the objective is a **single-basin concave program**: gradient methods find the global maximum every time — no bracketing, no multi-start, no policy flips.

2. **The contraction comes back.** The Bellman operator preserves concavity, so an ICNN value function keeps T a β -contraction on the relevant space. **Geometric convergence is restored**, at the classical rate.

3. **Prices behave.** Convex preferences give monotone demand, hence a unique, stable market-clearing price in a calibration loop.

Read benefit 2 again

7.B2's comparison table had one genuinely alarming row: *no general convergence guarantee*. **Imposing the shape the theory already gives you is what buys it back.** That

is a far better reason to use an ICNN than accuracy.

The cost is real and measured: the constrained network fitted $7.8\times$ less accurately here. You are buying a *guarantee* with *fit* — and for a Bellman solver that is a good trade.

Partial Convexity, Because Macro Is Not That Tidy

A value function $V(k, z)$ is typically:

- **concave in the endogenous states** k — capital, assets;
- **of no particular shape in the exogenous states** z — TFP, income risk.

A plain ICNN would impose convexity on z too. The **partially input convex** network splits the inputs:

$$z_{l+1} = \sigma \left(\underbrace{W_l^{(z)} z_l}_{\geq 0} + \underbrace{W_l^{(u)} (u \circ v)}_{\text{interaction}} + \underbrace{W_l^{(v)} v}_{\text{free}} + b_l \right),$$

with k down the constrained path and z down an ordinary one.

The general principle

Impose the restriction **exactly where theory gives you one**, and nowhere else. Over-imposing is bias by another route — and unlike under-imposing, it does not show up in the residual.

Softplus, not clamp

Enforce $W^{(z)} \geq 0$ by **reparameterising**, $W = \text{softplus}(\tilde{W})$, never by clamping: a clamped weight at the boundary gets **zero gradient** and is dead. 7.A2's dying ReLU by another door.

Monotone Networks, and the Conflict They Create

The same device gives monotonicity. If every weight along a path from input x_j to the output is non-negative and the activations are non-decreasing, then $\partial f / \partial x_j \geq 0$ **identically**.

Partial monotonicity is what is usually wanted: increasing in wealth, unrestricted in the shock. Constrain the signs only on the paths from the monotone inputs.

Cases where theory says exactly this: consumption increasing in cash on hand, saving in wealth, demand decreasing in own price, the value function increasing in the state.

The optimization conflict

Sign constraints **shrink the parameter set** and bend the loss surface, so a constrained network often trains more slowly and to a *higher* training loss. **Not a defect:** the unconstrained network reaches its lower loss partly by using functions the theory forbids.

Two routes: **constrain** the architecture (exact, costs flexibility) or **penalise** violations in the loss (flexible, only approximate).

The Instrument You Gave Up

3.A2 proved that value-function iteration is a contraction, and handed you a bound that needs no knowledge of the answer:

$$\|V_n - V^*\|_\infty \leq \frac{\beta^n}{1 - \beta} \|V_1 - V_0\|_\infty.$$

You could stop when it was small enough, and **know** how wrong you were.

No training loss replaces it. That number is measured on the states the optimizer just drew, and 7.A3's whole regularisation discipline exists because it does not mean what it appears to mean.

So the honest question is: **what *can* be measured?** — and the answer, next frame, is a bound that needs no training loss at all.

Three things a small loss is consistent with

1. A policy that is excellent where you sampled and arbitrary elsewhere — 7.B1 measured two decades lost at the band edge.
2. A policy satisfying a *necessary* condition on an explosive path.
3. An actor and a critic that agree with each other and with nothing else.

The Off-Sample Euler Residual

The workhorse diagnostic, and the one every number in this course uses. After training, evaluate

$$\log_{10} |\mathcal{R}(k_i, z_i)|, \quad i = 1, \dots, M,$$

on states **drawn independently of training** — a fine grid, or a fresh sample, and deliberately including the edges.

It is **unit-free**: divided by $u'(c)$, it reads as the relative consumption error a household makes by following this policy — -3 is one part in a thousand.

$\log_{10} \mathcal{R} < -3$	acceptable
$\log_{10} \mathcal{R} < -5$	good
$\log_{10} \mathcal{R} < -7$	excellent

The benchmark scale is Judd (1998), *Numerical Methods in Economics*.

Why it is the right instrument

Computed from the **model's own equilibrium conditions**, it needs no true solution to compare against — exactly the situation you are in when no classical method can solve the model. **And it is a bound, not a proxy.** Santos

(2000): the policy error is at most a constant times the maximum residual, the constant set by the curvature of u . 3.A1 states it; 5.B1 measured it.

Report the maximum, not the mean. A mean hides a failure concentrated in part of the region: 7.B1's mean was -3.43 and its maximum -1.61 .

Measured: This Session's Solutions on Judd's Scale

Method	$\log_{10} \max \mathcal{R} $, off-sample		verdict
	whole band	near k^*	
Chebyshev collocation (5.B6)	−6.40	−6.82	good, nearly excellent
perturbation, 2nd order (5.A6)	−4.50	−6.14	acceptable; good near k^*
global VFI (4.B5)	−2.51	−2.82	below acceptable
neural network (7.B1)	−1.61	−3.25	below acceptable on the band
<i>interior 90% only</i>	−2.54	—	still below −3

Say it plainly: our network does not reach Judd's “acceptable” across the band. It reaches −3.25 near the steady state and −1.61 at the edge.

Which is what reporting the number is for. The published figure for this kind of solver is −3.5 to −5; ours is worse, and a slide that quoted the literature instead of the run would have hidden that.

Ledger five_methods, worked_nn_growth; classical rows read from s05_numbers.json. Code: [s07_generate.py](#), [s05_generate.py](#).

Why Run It Anyway

At $d = 1$ the network spent thousands of parameters and minutes of compute to come **last** — the *expected* answer, not a disappointment.

A network is not a more accurate approximator. It is a differently scaling one. 7.A2 measured it with the parameter counts matched:

$\max \text{err} $	Chebyshev 17 coef.	network 16 par.
smooth target	2.5×10^{-11}	2.5×10^{-1}
kinked target	1.8×10^{-2}	2.0×10^{-4}

So the rule is not “networks are worse”. It is: **if a classical method can solve your model, use it** — and reach for the network when the state is a distribution, or the function has kinks you cannot locate in advance. S8 is entirely the first case.

Where the crossover is

7.A2's rates say it plainly: a basis fixed in advance is capped at $O(m^{-2/d})$ — the exponent carries d — while Barron's rate for a network does **not**. So the gap closes as d grows, then reverses. **The crossover is not at $d = 1$; it is out where a basis can no longer be written down.**

And the same reversal appears at $d = 1$ the moment the function is **kinked** — which is what a borrowing limit does to a policy.

The Benchmark Hierarchy

Validate against the strongest benchmark available, and be explicit about which one you had.

1. **A closed form.** The strongest, and rare — Brock–Mirman with $\delta = 1$. Tests *correctness*: a method that cannot recover $k' = \alpha\beta k^\alpha$ has a bug.
2. **A trusted classical solution.** 5.B6's Chebyshev solve, itself measured. Tests *accuracy* on a model with no closed form.
3. **The model's own conditions.** Off-sample Euler residuals. The only rung available at the frontier.
4. **Self-consistency.** Different seeds, widths, samplers agreeing. The weakest, and not nothing.

The discipline

Climb down the ladder, never up. Establish correctness on a model where rung 1 exists, then carry the *same code* to a model where only rung 3 does. This is what 1.A1 meant by grading a neural solver gate by gate against a classical solution before the government is allowed into the model.

S8 does exactly this: a neural heterogeneous-agent solver graded against a validated Aiyagari reference before any new economics is added.

What Even a Good Residual Will Not Catch

- **A necessary condition satisfied on the wrong path.**
The Euler residual says nothing about transversality. A small residual is compatible with an explosive path.
- **Shape violations that do not move the residual.** A value function with a small concave patch can have a fine residual and a broken maximisation.
- **Regions never visited.** The residual is only measured where you measured it.
- **A wrong equilibrium.** With multiple equilibria, every one of them has a zero residual.

So check the economics too

Simulate and look at the **ergodic distribution**: does capital settle where theory says? Check **shape** directly — the sign of the fitted second derivative on a grid, which is the number reported three frames back. Check **aggregates** against any moment you believe.

And vary the seed. A result that moves with the random seed is not a result — which is why width-stability across seeds is reported as evidence at all.

One Model, Five Methods — the Reprise

The arc closes where it opened. The same quarterly RBC model has now been solved by **grid VFI** (4.B5), **first- and second-order perturbation** (5.A6), **Chebyshev collocation** (5.B6), and **a neural network** (7.B1).

On this model the ranking is unambiguous and unflattering to the newest method. **Every column scales differently**, and that is the only reason the last row exists.

The rule to leave with: if a classical method can solve your model, use it. Reach for the network when the state is a distribution, or when d is large enough that a basis cannot be written down.

What the network did buy, even here

A mesh-free policy — 4.B5's grid quantisation is simply absent. **Feasibility by construction**, not by clipping. **A method whose cost in d is polynomial**, which is the only one of the five of which that is true.

And what it cost is narrower than it looks: the **a priori** guarantee, 3.A2's $\beta^n / (1 - \beta)$. What survives is the **a posteriori** bound — Santos (2000) — and, with shape constraints, the contraction itself.

Ledger `five_methods`. The comparison was the point of running the network on a problem it was always going to lose. Code: [s07_generate.py](#); L07.

Takeaways, and Where Session 8 Goes

- Theory gives shapes, and a generic network obeys none of them. **Measured over 5 seeds: a plain net fitted to an exactly convex function came out locally concave by 5.9 at the median** — and convex at one. Not reliably wrong: **unreliable**.
- An **ICNN** is convex at every seed and approximates *any* convex function — and fitted **7.8×** less accurately. **The guarantee has a price**.
- The deepest reason to impose it is not accuracy: **concavity restores the Bellman contraction** that 7.B2 gave up.
- Validation is the **off-sample** maximum Euler residual, on a known scale.

Bridge to Session 8

Every model in this session had a **small** state: k , or (k, z) . The networks were never needed — that was the point of measuring. **S8 removes the training wheels**. The state becomes a *distribution*; the transition $H(\Gamma)$ is no longer known; and the methods of this session become the only ones available. 7.A4 already named the bridge: **permutation invariance**.